

R Programming

Day 02 — From Data Structures to Functions

Logical operators • Lists • Matrices • Data frames • Flow control • Functions

Learning objectives

By the end of this session, you should be able to:

Use logical and comparison operators to build conditions.

Create, access, modify, combine, and filter lists.

Construct matrices and understand indexing, dimensions, and matrix operations.

Create and manipulate data frames—the core tabular structure used in R.

Control program flow with if, else if, else, for, while, and repeat.

Write reusable functions with arguments, return values, and local scope.

Today's roadmap

A practical progression from operators → data structures → program logic

01 Logical operators

Build conditions

02 Lists

Store mixed objects

03 Matrices

2D homogeneous data

04 Data frames

Tabular data

05 Flow control

Control execution

06 Functions

Reusable code

Logical operators

Logical operators combine or negate logical values (TRUE / FALSE).

Operator	Meaning	Example	Result
&&	AND — first element only	TRUE && FALSE	FALSE
	OR — first element only	TRUE FALSE	TRUE
&	AND — element-wise	c(TRUE,FALSE) & c(TRUE,TRUE)	TRUE FALSE
	OR — element-wise	c(TRUE,FALSE) c(FALSE,FALSE)	TRUE FALSE
!	NOT	!TRUE	FALSE

```
x <- 75
x >= 50 && x <= 100
# TRUE
```

```
score <- c(45, 72, 88)
score >= 50
# FALSE TRUE TRUE
```

Conditions in practice

Comparison operators produce logical values; logical operators combine them.

R

```
age <- 21  
score <- 82
```

```
age >= 18 && score >= 80  
# TRUE
```

```
score < 50 || age < 18  
# FALSE
```

Think in two steps

1. Ask a question with a comparison.
2. Combine questions with logical operators.

Common pattern

$x \geq \text{lower} \ \& \ x \leq \text{upper}$

Useful for filtering observations inside a range.

Practice

Write one condition that checks whether x is between 10 and 20 (inclusive).

Lists in R

A list can contain elements of different types, including vectors, strings, numbers, and other lists.

R

```
first_list <- list("apple", "banana", "cherry")  
first_list
```

```
first_list[[1]]  
# "apple"
```

```
first_list[[1]] <- "orange"  
first_list
```

Key idea

A list is a flexible container. Its elements do not have to share the same data type.

[] vs [[]]

`x[1]` → returns a sub-list
`x[[1]]` → returns the element itself

Named lists

```
student <- list(name="Asha", age=21, marks=c(80,85,90))
```

List operations

Create, inspect, add, remove, and combine list elements.

Task	R code	Purpose
Length	<code>length(first_list)</code>	Number of elements
Check membership	<code>"apple" %in% first_list</code>	TRUE / FALSE
Add	<code>append(first_list, "mango")</code>	Append an element
Remove	<code>first_list[-1]</code>	Drop first element
Range	<code>first_list[2:3]</code>	Select a range
Combine	<code>c(list1, list2)</code>	Join lists

```
list1 <- list("a","b")
list2 <- list(1,2)
list3 <- c(list1, list2)
```

Matrix in R

A matrix is a two-dimensional, homogeneous data structure.

R

```
m <- matrix(1:6, nrow = 2, ncol = 3)
```

```
m
```

```
#      [,1] [,2] [,3]  
# [1,]    1    3    5  
# [2,]    2    4    6
```

Important property

All elements of a matrix must have a compatible data type.

Dimensions

`dim(m) → c(2, 3)`

`nrow(m) → 2`

`ncol(m) → 3`

By row

`matrix(1:6, nrow=2, byrow=TRUE)`

Fills the matrix row-by-row.

Matrix indexing & operations

Use [row, column] indexing and standard matrix operators.

R

```
m <- matrix(1:9, nrow=3)
```

```
m[2, 3]      # row 2, column 3
```

```
m[1, ]      # first row
```

```
m[, 2]      # second column
```

```
m[1:2, 2:3] # sub-matrix
```

Operation	R syntax	Meaning
Element-wise multiply	$A * B$	Multiply corresponding elements
Matrix multiply	$A \%*\% B$	Linear algebra product
Transpose	$t(A)$	Swap rows and columns
Bind rows	$rbind(A,B)$	Stack by rows
Bind columns	$cbind(A,B)$	Join by columns

Practice

Create a 2×3 matrix containing 10, 20, 30, 40, 50, 60 and print its second column.

Data frames

The standard R structure for rectangular, tabular data.

Rows represent observations; columns represent variables.

Columns can have different data types (numeric, character, logical, factor, etc.).

All columns must have the same number of rows.

Data frames are the main structure you will meet in statistical analysis.

R

```
students <- data.frame(  
  name = c("Asha", "Rafi", "Nila"),  
  age  = c(21, 22, 20),  
  score = c(82, 74, 91)  
)  
students
```

Working with data frames

The core operations you will use in almost every analysis.

Goal	R code	What it does
Inspect	<code>head(students)</code>	First rows
Structure	<code>str(students)</code>	Types + structure
Dimensions	<code>dim(students)</code>	Rows and columns
Column	<code>students\$score</code>	Select one column
Row + column	<code>students[2, 3]</code>	Specific cell
Filter	<code>students[students\$score >= 80,]</code>	Keep matching rows
Add column	<code>students\$pass <- students\$score >= 50</code>	Create a variable

```
students[students$score >= 80, c("name", "score")]
```

Flow control

Flow control determines which statements run, and how often they run.

if / else

Choose between paths

else if

Test multiple conditions

for

Repeat over a sequence

while

Repeat while a condition is TRUE

repeat

Repeat until explicitly stopped

break / next

Stop or skip an iteration

if, else if, else

Use multiple branches when more than one condition is possible.

R

```
score <- 76

if (score >= 80) {
  grade <- "A"
} else if (score >= 70) {
  grade <- "B"
} else {
  grade <- "C or below"
}

print(grade)
```

Execution rule

R checks conditions from top to bottom. The first TRUE branch is executed.

Good practice

Keep conditions mutually clear and readable. Use braces for multi-line blocks.

Challenge

Modify the example to create grades A+, A, B, C, and F.

for loop

Repeat a block once for each value in a sequence.

R

```
for (i in 1:5) {  
  print(i)  
}
```

```
# 1  
# 2  
# 3  
# 4  
# 5
```

R

```
scores <- c(72, 85, 64)  
  
for (s in scores) {  
  if (s >= 70) print("Pass")  
  else print("Needs improvement")  
}
```

Pattern

```
for (item in sequence) { statements }
```

while & repeat

Use while when the stopping condition is known; repeat when you want explicit control.

R

```
i <- 1
while (i <= 5) {
  print(i)
  i <- i + 1
}
```

R

```
i <- 1
repeat {
  print(i)
  i <- i + 1
  if (i > 5) break
}
```

Important

A while/repeat loop can become infinite if its stopping condition is never reached.

break & next

Control what happens inside a loop.

R

```
for (i in 1:5) {  
  if (i == 3) next  
  print(i)  
}  
# 1 2 4 5
```

R

```
for (i in 1:5) {  
  if (i == 3) break  
  print(i)  
}  
# 1 2
```

next

Skip the current iteration and continue the loop.

break

Stop the loop immediately.

Functions in R

A function packages instructions into a reusable unit.

R

```
function_name <- function(argument1, argument2) {  
  # statements  
  result <- argument1 + argument2  
  return(result)  
}
```

Why functions?

- Reduce repeated code
- Improve readability
- Make analysis easier to test
- Accept different inputs

Core structure

```
function(arguments) { body }
```

The body executes when the function is called.

Writing and calling a function

Arguments make the same function useful for many inputs.

R

```
mean_two <- function(x, y) {  
  (x + y) / 2  
}
```

```
mean_two(10, 20)  
# 15
```

```
mean_two(7, 13)  
# 10
```

Implicit return

R automatically returns the value of the last evaluated expression.

Explicit return

`return(value)` is useful when you want to exit early or make the returned value explicit.

Default argument

```
power <- function(x, p = 2) x^p  
power(5) → 25
```

Arguments & scope

Understanding local variables prevents unexpected results.

R

```
add_tax <- function(price, rate = 0.05) {  
  tax <- price * rate  
  price + tax  
}
```

```
add_tax(100)  
# 105
```

```
add_tax(100, 0.10)  
# 110
```

Arguments

Inputs supplied to a function. Defaults can be provided, e.g. `rate = 0.05`.

Local scope

Variables created inside a function normally exist only inside that function.

Design principle

Prefer passing values as arguments and returning results rather than relying on global variables.

Mini practice

Try these before moving to the next session.

01 • Logical

Write a condition that returns TRUE when x is between 50 and 100.

02 • Matrix

Create a 3×3 matrix and print its second row.

03 • Data frame

Create a data frame with name, age, and score; filter score ≥ 80 .

04 • Loop

Print the even numbers from 2 to 10 using a for loop.

05 • Function

Write a function that takes x and returns $x^2 + 2x + 1$.

06 • Challenge

Combine a function + condition to classify a score as Pass/Fail.

```
# Suggested workflow
# 1. Write the smallest working version.
# 2. Test with at least two inputs.
# 3. Check the output before moving on.
```

Key takeaways

The six ideas from Day 02 form the foundation for practical R programming.

Logical operators let you build precise conditions from TRUE/FALSE values.

Lists store flexible, potentially mixed-type collections.

Matrices store homogeneous two-dimensional data and support matrix operations.

Data frames organize heterogeneous variables in rows and columns.

Flow control lets a program branch and repeat based on conditions.

Functions turn repeated logic into reusable, testable components.

Next step

Practice each structure in RStudio. Then use them together on a small dataset.

Questions & Discussion

Bring your code, mistakes, and questions.